DLDCA Lab 4: Simulator
Weeks 2 & 3: Report

Swara Dipak Takale, 24B0972

November 20, 2025

1 Task 1

```
void CACHE::handle_fill()
  while (writes_available_this_cycle > 0) {
    if (way == NUM_WAY) {
      way = impl_replacement_find_victim(fill_mshr->cpu, fill_mshr->instr_id, set, &block.data()[set
                                         fill_mshr->type);
    }

    #ifdef DEBUG_TRACK
      trackPkt(*fill_mshr,NAME,"handle_fill",false,true,true,true);
      trackaddr(fill_mshr->address, NAME, "handle_fill");
    #endif
    ////////////// start change ////////////////////////////////////////
    if(check_string(NAME,"LLC")){
      sim_miss[fill_mshr->cpu][fill_mshr->type]++;
      sim_access[fill_mshr->cpu][fill_mshr->type]++;
      if (warmup_complete[fill_mshr->cpu] && (fill_mshr->cycle_enqueued != 0))
      total_miss_latency += current_cycle - fill_mshr->cycle_enqueued;
      MSHR.erase(fill_mshr);
      writes_available_this_cycle--;
      continue;
    }
    /////////// end change ////////////////////////////////////////////
    bool success = filllike_miss(set, way, *fill_mshr);
    if (!success) {
      return;
    }

    if (way != NUM_WAY) {
      // update processed packets
```

Figure 1: my code for handle_fill

What is the code difference?
In handle_fill function added an if block which checks if there's a response to LLC from DRAM and updates the miss ,access and miss latency stats, erases the data from MSHR and decrements the writes available in that cycle and proceeds with the next iteration.

Why did you make this change?
Since we are implementing an exclusive LLC, when a response comes from the DRAM, we do not fill into the LLC, because the memory address will anyway be filled into the L1D and L2 Caches. handle_fill() function handles responses, therefore, for LLC cache we make the changes in the handle_fill() function.

```
void CACHE::handle_read()
  while (reads_available_this_cycle > 0) {
    #ifdef DEBUG_TRACK
    #endif

    // A (hopefully temporary) hack to know whether to send the evicted paddr or
    // vaddr to the prefetcher
    ever_seen_data |= (handle_pkt.v_address != handle_pkt.ip);

    uint32_t set = get_set(handle_pkt.address);
    uint32_t way = get_way(handle_pkt.address, set);

    if (way < NUM_WAY) // HIT
    {
      readlike_hit(set, way, handle_pkt);
      ////////////////// start change ////////////////////////////////
      if(check_string(NAME,"LLC")) invalidate_entry(handle_pkt.address);
      ////////////////// end change //////////////////////////////////
    } else {
      bool success = readlike_miss(handle_pkt);
      if (!success)
        return;
    }

    // remove this entry from RQ
    RQ.pop_front();
    reads_available_this_cycle--;
  }
}
```

Figure 2: my code for handle_read

What is the code difference?
In handle_read function added an if block after a hit, which checks if the cache is LLC and invalidates the entry that got a hit.

Why did you make this change?
We will only get a LLC hit if the L2 cache has requested for that memory address, meaning the memory will be filled into the L2, thus, it should be removed from the LLC. handle_read() function handles requests, therefore, for a hit in LLC we invalidate the entry in the function.

```
bool CACHE::filllike_miss(std::size_t set, std::size_t way, PACKET& handle_pkt)
  bool evicting_dirty = !bypass && (lower_level != NULL) &&  fill_block.dirty;
  /////////////// start change ////////////////////////////
  evicting_dirty = evicting_dirty || check_string(NAME,"L2C");
  ////////////// end change ////////////////////////////////
  uint64_t evicting_address = 0;

#ifdef DEBUG_TRACK
  trackaddr(handle_pkt.address, NAME, "filllike_miss");
#endif
  if (!bypass) {
    if (evicting_dirty) {
      PACKET writeback_packet;

      writeback_packet.fill_level = lower_level->fill_level;
      writeback_packet.cpu = handle_pkt.cpu;
      writeback_packet.address = fill_block.address;
      writeback_packet.data = fill_block.data;
      writeback_packet.instr_id = handle_pkt.instr_id;
      writeback_packet.ip = 0;
      writeback_packet.type = WRITEBACK;
      writeback_packet.l2Hit = fill_block.l2Hit;
      writeback_packet.from_llc = fill_block.from_llc;
      ////////// start change ////////////////////
      writeback_packet.dirty = fill_block.dirty;
      /////////// end change /////////////////////
      auto result = lower_level->add_wq(&writeback_packet);
      if (result == -2)
        return false;
```

Figure 3: my code for filllike_miss

What is the code difference?
The evicting_dirty boolean variable now also checks the condition if the cache is L2C. And the dirty blocks are now being marked as dirty when we evict them.

Why did you make this change?
The L2 Cache must writeback clean blocks to the LLC: This is because otherwise clean blocks will never get filled into the LLC. By default, only dirty blocks are written back by the L2 to the LLC, therefore, the variable evicting_dirty checking dirty blocks, now also checks if the block being evicted is from L2C cache, no matter if it is clean or dirty and we send the writeback packet to the lower level. We also need to ensure that the dirty blocks are being marked as dirty so that the correct changes can be made in lower levels.

2 Task 2

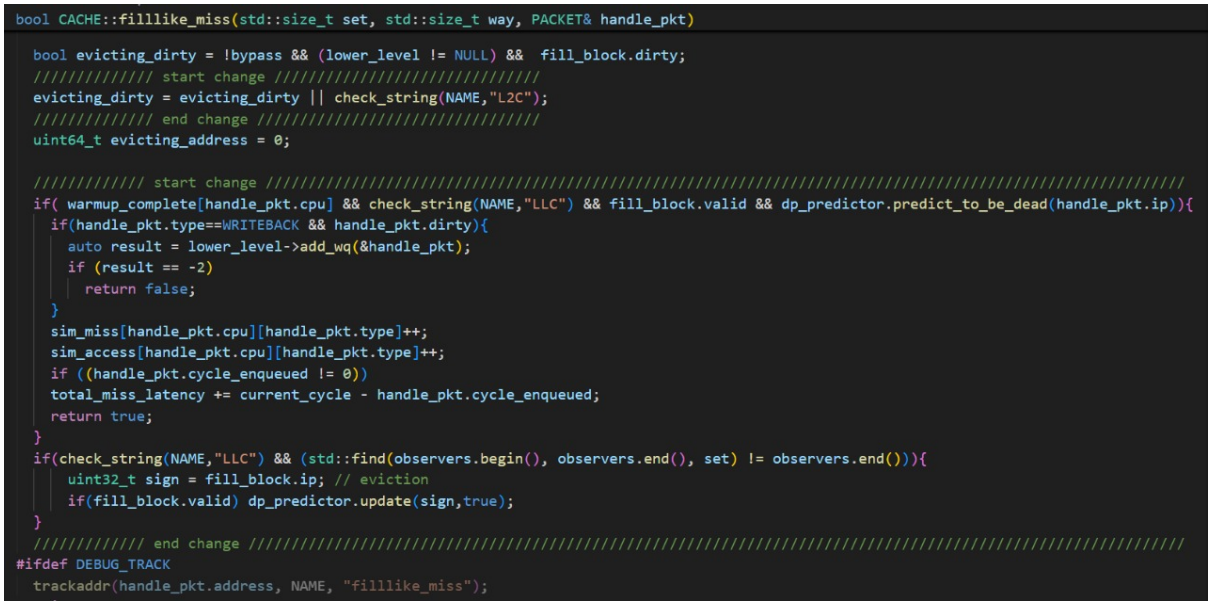
Since task 2 is done in an exclusive heirarchy, the changes from task 1 persist. Further changes for task 2 are explained below:

```
void CACHE::readlike_hit(std::size_t set, std::size_t way, PACKET& handle_pkt)
  DP(if (warmup_complete[handle_pkt.cpu]) {
    std::cout << " full_addr: " << handle_pkt.address;
    std::cout << " full_v_addr: " << handle_pkt.v_address << std::dec;
    std::cout << " type: " << +handle_pkt.type;
    std::cout << " cycle: " << current_cycle << std::endl;
  });

  ////////////// start change ////////////////////////////////////////////////////////////////////////
  if(check_string(NAME,"LLC") && (std::find(observers.begin(), observers.end(), set) != observers.end())){
    uint32_t sign = block[set * NUM_WAY + way].ip;
    if(block[set * NUM_WAY + way].valid) dp_predictor.update(sign,false);
  }
  ////////////// end change //////////////////////////////////////////////////////////////////////////
  BLOCK& hit_block = block[set * NUM_WAY + way];

  handle_pkt.data = hit_block.data;
  // update prefetcher on load instruction
  if (should_activate_prefetcher(handle_pkt.type) && handle_pkt.pf_origin_level < fill_level) {
    cpu = handle_pkt.cpu;
    uint64_t pf_base_addr = (virtual_prefetch ? handle_pkt.v_address : handle_pkt.address) & ~bitmask(match_
    handle_pkt.pf_metadata = impl_prefetcher_cache_operate(pf_base_addr, handle_pkt.ip, 1, handle_pkt.type,
  }
```

Figure 4: my code for readlike_hit

What is the code difference?
In readlike_hit() function, added an if block to check if the cache is LLC and whether the set is one of the observer sets. If that condition is true then, for a valid block, call the update function of the dp_predictor.

Why did you make this change?
When there is a hit in LLC, we need to decrement the counters for the PC of the instruction that brought this block in cache, as it is considered live. But it is sufficient to only update this information if the set is in observers.



```
bool CACHE::filllike_miss(std::size_t set, std::size_t way, PACKET& handle_pkt)

  bool evicting_dirty = !bypass && (lower_level != NULL) &&  fill_block.dirty;
  ////////////// start change ////////////////////////////
  evicting_dirty = evicting_dirty || check_string(NAME,"L2C");
  ////////////// end change //////////////////////////////
  uint64_t evicting_address = 0;

  ////////////// start change ////////////////////////////////////////////////////////////////////////////////////////
  if( warmup_complete[handle_pkt.cpu] && check_string(NAME,"LLC") && fill_block.valid && dp_predictor.predict_to_be_dead(handle_pkt.ip)){
    if(handle_pkt.type==WRITEBACK && handle_pkt.dirty){
      auto result = lower_level->add_wq(&handle_pkt);
      if (result == -2)
        return false;
    }
    sim_miss[handle_pkt.cpu][handle_pkt.type]++;
    sim_access[handle_pkt.cpu][handle_pkt.type]++;
    if ((handle_pkt.cycle_enqueued != 0))
    total_miss_latency += current_cycle - handle_pkt.cycle_enqueued;
    return true;
  }
  if(check_string(NAME,"LLC") && (std::find(observers.begin(), observers.end(), set) != observers.end())){
      uint32_t sign = fill_block.ip; // eviction
      if(fill_block.valid) dp_predictor.update(sign,true);
  }
  ////////////// end change ////////////////////////////////////////////////////////////////////////////////////////
#ifdef DEBUG_TRACK
  trackaddr(handle_pkt.address, NAME, "filllike_miss");
```

Figure 5: my code for filllike_miss

What is the code difference?

The change in the variable evicting_dirty is the same as in task 1 for the same reasons provided in task 1.

Further added an if block which checks whether the block being filled in LLC is a deadblock. If it is deadblock, then we update the miss, access and miss latency stats, and do not fill it in the LLC but we also check if the block was of type writeback and was dirty so that we can send the dirty data to DRAM. Further added an if block to check if the cache is LLC and whether the set is one of the observer sets. If it is then, for a valid block, call the update function of the dp_predictor.

Why did you make this change?

If a block being filled in LLC is predicted to be dead then we can boost our perfomance by not filling the block in the cache, because it will be most likely evicted before getting a hit. But we need to be careful about the dirty writeback blocks, since they are evicted from higher level caches and won't be filled in the LLC, we need to send the dirty writeback packets to lower level i.e. DRAM so it is correctly saved in the memory.

In LLC, if a valid block has been chosen as a victim to be replaced, then the block is dead and we need to increment the counters for the PC of the instruction that brought this block in cache. But it is sufficient to only update this information if the set is in observers.

```
bool CACHE::filllike_miss(std::size_t set, std::size_t way, PACKET& handle_pkt)
  if (!bypass) {

    if (evicting_dirty) {
      PACKET writeback_packet;

      writeback_packet.fill_level = lower_level->fill_level;
      writeback_packet.cpu = handle_pkt.cpu;
      writeback_packet.address = fill_block.address;
      writeback_packet.data = fill_block.data;
      writeback_packet.instr_id = handle_pkt.instr_id;
      // original : writeback_packet.ip = 0;
      ////////// start change /////////////////
      writeback_packet.ip = fill_block.ip;
      ///////// end change ///////////////////
      writeback_packet.type = WRITEBACK;
      writeback_packet.l2Hit = fill_block.l2Hit;
      writeback_packet.from_llc = fill_block.from_llc;
      ///////// start change ///////////////////
      writeback_packet.dirty = fill_block.dirty;
      ///////// end change /////////////////////
      auto result = lower_level->add_wq(&writeback_packet);
      if (result == -2)
        return false;
    }
```

Figure 6: my code for filllike_miss

What is the code difference?
In filllike_miss() function while sending the writeback packet, the PC of writeback packet is assigned as the PC of the block that is being replaced, instead of 0. The other change is the same as task 1 for the same reasons provided in task 1.

Why did you make this change?
Since we need to build PC counter table to maintain dead - live block counts, every block must carry the information about the PC of the instruction that brought that block in the cache so that we can update the counter table for the PC whenever needed (on hit/eviction) and predict if any block being filled in LLC is brought by the instruction having a PC that mostly brings in deadblocks, so that we can avoid filling them and boost performance. In the original code the writeback packet didn't carry this information, hence the change.